```
# MNIST 手写识别项目二
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
# 数据加载
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_dataset = datasets.MNIST(root='./data', train=True, download=True,

transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True,

transform=transform)
train_loader    =    torch.utils.data.DataLoader(train_dataset,    batch_size=64,
shuffle=True)
test_loader    =    torch.utils.data.DataLoader(test_dataset,    batch_size=64,
shuffle=False)
# 卷积核的通道数必须与输入通道数一致
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # 1. 第一个卷积层：输入通道 1，输出通道 32，卷积核 5x5
        # 2. 每一个卷积核的通道数必须等于输入通道数（这里是 1）
        # 3. 输出通道数 32 = 卷积核的数量
        self.conv1 = nn.Conv2d(1, 32, kernel_size=5)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=5)
        # 经过两次卷积池化后，特征图尺寸从 28x28 → 24x24 → 12x12 → 8x8 → 4x4
        self.fc1 = nn.Linear(64 * 4 * 4, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(2)
    def forward(self, x):
        # 卷积 → 激活 → 池化
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 64 * 4 * 4)
        x = self.relu(self.fc1(x))
```

```
        x = self.fc2(x)
        return x
model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# 训练
epochs = 5
for epoch in range(epochs):
    running_loss = 0.0
    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f'Epoch {epoch + 1}, Loss: {running_loss / len(train_loader):.4f}')
# 测试
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()
        total += target.size(0)
print(f'CNN Test Accuracy: {100 * correct / total:.2f}%')
```